HeatLoad HVAC Calculation Tool

Comprehensive Project Documentation

Version 2.8 — June 2026

Framework:	React 19 + Redux Toolkit + Vite 7
Language:	TypeScript / JavaScript (JSX/TSX)
Styling:	Tailwind CSS v4
Deployment:	Vercel (SPA)
Standards:	ASHRAE HOF 2021, ASHRAE 62.1-2022, ISO 14644-1:2015
Industries:	Semiconductor, Pharmaceutical, Battery, Solar

Generated automatically from source code analysis.

Contents

Chapter 1

Project Overview

1.1 Introduction

The **HeatLoad HVAC Calculation Tool** is a client-side, single-page application (SPA) designed for HVAC engineers working in critical-environment industries—semiconductor fabrication, pharmaceutical manufacturing, battery manufacturing (Li-ion and lead-acid), and solar/PV production. The application automates the full ASHRAE Handbook of Fundamentals (2021) Chapter 18 heat load calculation methodology, from building envelope analysis through psychrometric state-point resolution.

The tool replaces traditional Excel-based Room Data Sheet (RDS) calculators with a reactive, browser-based interface that provides instant recalculation as engineers modify room parameters, climate data, or system design assumptions.

1.2 Key Engineering Capabilities

- **Multi-Season Load Calculation** — Summer, monsoon, and winter seasons with peak-season auto-selection for both CFM (supply air) and TR (cooling capacity).
- **CLTD/CLF Envelope Method** — Walls, roofs, glazing, skylights, partitions, and floors with ASHRAE construction presets.
- **Psychrometric Engine** — Hyland-Wexler saturation pressure (ASHRAE HOF 2021 Ch.1, Eq. 3 & 5) replacing the less-accurate Magnus approximation.
- **ASHRAE 62.1-2022 Ventilation** — Full Ventilation Rate Procedure (VRP) with industry-specific categories including lead-acid battery rooms (OSHA 29 CFR 1926.403(i)).
- **ISO 14644-1:2015 Classification** — ISO 1 through ISO 9, CNC, and Unclassified with IEST-RP-CC012.2 ACPH ranges.
- **GMP Annex 1:2022 Compliance** — Grade A through D mapping with mandated ACPH floors and 1.25× process safety factors.
- **Altitude-Corrected Psychrometrics** — All factors (C_s, C_l, humidity ratio) corrected via the barometric formula to actual site elevation.

- **Reheater Sizing** — Physics-based minimum ESHF calculation and ASHRAE Ch.18 §17.3 reheat load determination.
- **Required ADP Analysis** — ESHF line / saturation curve intersection with three-outcome classification (`found`, `sensible_only`, `no_solution`).
- **CHW/HW Pipe Sizing** — Automated pipe diameter selection with velocity-based branch and manifold sizing.
- **Humidification Sizing** — Winter moisture deficit calculation with desiccant system awareness for battery dry rooms.

1.3 Technology Stack

Table 1.1: Technology Stack Summary

Technology	Purpose
React 19.2	UI component framework
Redux Toolkit 2.11	Centralised state management with Immer-powered immutable updates
React Router DOM 7.13	Client-side routing (SPA)
Vite 7.3	Build tooling, HMR dev server
Tailwind CSS 4.2	Utility-first CSS framework
TypeScript 6.0	Static type checking
Vitest 4.1	Unit testing framework
ExcelJS 4.4	Excel export (`.xlsx`) generation
Lucide React	Icon library
React Hot Toast	Notification toasts
Vercel	Deployment platform with SPA rewrite rules

Chapter 2

Architecture

2.1 High-Level Architecture

The application follows a **unidirectional data flow** pattern:

1. **User interaction** dispatches Redux actions.
2. **Redux reducers** (slices) produce new immutable state.
3. **Memoised selectors** (`createSelector`) derive computed RDS data.
4. **React components** re-render with the new data.

All calculation logic is **pure**—implemented as selector modules in `features/results/`. These modules are never registered as Redux reducers; they derive from the five state slices and produce the full RDS row object on every state change.

2.2 State Shape

The Redux store is composed of five independent slices:

```
rootReducer = {
  project:  projectReducer,   // Project metadata, ambient, system design
  ahu:      ahuReducer,       // Air Handling Unit definitions
  room:     roomReducer,      // Room list and active selection
  climate:  climateReducer,   // Outdoor design conditions (3 seasons)
  envelope: envelopeReducer,  // Per-room building envelope + internal loads
}
```

Listing 2.1: Root Reducer Composition

2.3 Data Flow Diagram

```
UI Input
   |
```

```
    v
Redux Dispatch (action)
    |
    v
Slice Reducer (state update)
    |
    v
selectRdsData (createSelector, memoised)
    |
    +-- calculateSeasonLoad()     [seasonalLoads.ts]
    +-- calculateAirQuantities() [airQuantities.ts]
    +-- calculateAllSeasonOALoads() [outdoorAirLoad.ts]
    +-- calculateRequiredADP()    [psychro.ts]
    +-- calculateHeatingHumid()   [heatingHumid.ts]
    +-- calculatePipeSizing()     [pipeSizing.ts]
    +-- calculateAllSeasonStatePoints() [psychroStatePoints.ts]
    |
    v
RDS Row Object (one per room)
    |
    v
React Component Tree (re-render)
```

2.4 Directory Structure

```
client/src/
  app/
    store.ts            -- Redux store configuration
    rootReducer.ts      -- Slice composition
  components/
    Layout/             -- Header, TabNav, RoomSidebar
    UI/                 -- InputField, InputGroup, NumberControl,
     StatCard
  constants/
    ashrae.ts           -- ASHRAE constants & conversion factors
    ashraeTables.ts     -- CLTD/CLF lookup tables, U-value presets
    isoCleanroom.ts     -- ISO 14644 classification data
    ventilation.ts      -- ASHRAE 62.1-2022 VRP implementation
  features/
    project/projectSlice.ts   -- Project, ambient, system design
     state
    ahu/ahuSlice.ts           -- AHU definitions
    room/roomSlice.ts         -- Room list & geometry
    room/roomActions.js       -- Cross-slice room thunks
    room/AhuAssignment.jsx    -- AHU-to-room assignment UI
    climate/climateSlice.ts   -- Seasonal outdoor conditions
    envelope/envelopeSlice.ts -- Building elements + internal loads
    envelope/BuildingShell.tsx -- Envelope editor component
    results/
      rdsSelector.ts          -- Master RDS orchestrator
      seasonalLoads.ts        -- Per-season load calculation
      airQuantities.ts        -- All CFM quantities
```

```
      outdoorAirLoad.ts        -- OA coil loads
      heatingHumid.ts          -- Heating + humidification
      pipeSizing.ts            -- CHW/HW pipe diameter selection
      psychroStatePoints.ts    -- AHU air stream state points
  hooks/
    useActiveRoom.js           -- Active room selection hook
    useNumberControl.ts        -- Numeric input stepper hook
    useProjectTotals.js        -- Project-wide aggregation hook
    useRdsRow.js               -- Single RDS row data hook
    useRoomSidebar.ts          -- Sidebar navigation hook
  pages/
    Home.jsx                   -- Landing page
    ProjectDetails.jsx         -- Project metadata entry
    AHUConfig.jsx              -- AHU equipment configuration
    RoomConfig.jsx             -- Room geometry & classification
    ClimateConfig.jsx          -- Outdoor design conditions
    EnvelopeConfig.jsx         -- Building envelope editor
    RDSPage.jsx                -- Room Data Sheet spreadsheet view
    ResultsPage.jsx            -- Project results & export
    rds/                       -- RDS sub-components
  utils/
    psychro.ts                 -- Psychrometric calculations
    units.ts                   -- Unit conversions
    types.ts                   -- TypeScript interfaces
    envelopeCalc.ts            -- CLTD/CLF envelope calculations
    envelopeAggregator.ts      -- Envelope gain summation
    envelopeHelpers.ts         -- Envelope utility functions
    glazingCalc.ts             -- Glass & skylight solar gains
    isoValidation.ts           -- ISO/GMP compliance validation
    psychroValidation.ts       -- Psychrometric consistency checks
```

Listing 2.2: Source Directory Layout

Chapter 3

Entry Points & Application Shell

3.1 client/src/main.jsx — Application Bootstrap

The application entry point. Renders the React root with:

- StrictMode — Enables development-time double-rendering for side-effect detection.
- Provider store={store} — Makes the Redux store available to all descendant components via useSelector / useDispatch.
- App — The root application component (routing).

3.2 client/src/App.jsx — Routing & Layout

Defines the application's route structure using React Router v7:

Table 3.1: Application Routes

Path	Component	Description
/	Home	Public landing page (no header/nav)
/project	ProjectDetails	Project metadata and system design
/rds	RDSPage	Room Data Sheet spreadsheet
/ahu	AHUConfig	AHU equipment configuration
/room	RoomConfig	Room geometry and classification
/climate	ClimateConfig	Outdoor design conditions
/envelope	EnvelopeConfig	Building envelope editor
/results	ResultsPage	Results summary and export
*	Redirect to /	Catch-all

The AppLayout wrapper provides the fixed-viewport layout: Header + TabNav at natural height, <main> fills remaining viewport via flex-1 overflow-auto min-h-0.

3.3 `client/src/app/store.ts` — Redux Store

Configures the Redux store with:

- Root reducer composed from five slices.
- Dev tools enabled in non-production mode.
- Serializable check with `climate` slice ignored (transient `NaN` during numeric input).
- Automatic type inference for `RootState` and `AppDispatch`.
- Global `window.__store` binding in development for console debugging.

Chapter 4

Redux State Slices

4.1 `features/project/projectSlice.ts` — Project State

4.1.1 State Shape

```
state.project = {
  info: {
    projectName, projectLocation, customerName,
    consultantName, industry, keyAccountManager
  },
  ambient: {
    elevation,         // ft (0 = sea level)
    latitude,          // degrees (-90..90); default 28 (Delhi)
    dailyRange,        // F daily DB swing
    dryBulbTemp,       // C (reference only, NOT used in calcs)
    wetBulbTemp,       // C (reference only)
    relativeHumidity   // % (reference only)
  },
  systemDesign: {
    safetyFactor,            // % (default 10)
    ductHeatGain,            // % (default 5)
    bypassFactor,            // dimensionless (default 0.10)
    adp,                     // F (default 55)
    adpMode,                 // 'manual' | 'calculated'
    fanHeat,                 // % (default 5)
    returnFanHeat,           // % (default 5)
    humidificationTarget     // %RH (default 45)
  }
}
```

Listing 4.1: Project State Shape

4.1.2 Reducers

- `updateProjectInfo({field, value})` — Updates project metadata string fields with `trim()`.
- `updateAmbient({field, value})` — Parses, clamps to `AMBIENT_BOUNDS`, and stores numeric ambient values.

- `updateSystemDesign({field, value})` — String fields (e.g. `adpMode`) bypass `parseFloat`; numeric fields are clamped to `SYSTEM_DESIGN_BOUNDS`.
- `resetProject()` — Returns entire slice to `initialState`.

4.1.3 Selectors

14 exported selectors provide targeted access: `selectProjectInfo`, `selectAmbient`, `selectSystemDesign`, `selectElevation`, `selectLatitude`, `selectDailyRange`, `selectSafetyFactor` `selectDuctHeatGain`, `selectBypassFactor`, `selectAdp`, `selectFanHeat`, `selectReturnFanHeat`, `selectHumidTarget`, `selectIndustry`.

4.1.4 Key Design Decisions

- **ADP Scope:** Project ADP is the default; per-AHU overrides live in `ahuSlice`. Priority chain: AHU calculated > AHU manual > project default > 55°F fallback.
- **Duct Heat Gain:** Applied *additively* with safety factor per Excel row 80 method: mult = 1 + (safety + duct)/100.
- **Ambient fields** (`dryBulbTemp`, `wetBulbTemp`, `relativeHumidity`) are brief reference values in °C — they are *not* used in load calculations. Seasonal design conditions live in `climateSlice`.

4.2 `features/ahu/ahuSlice.ts` — AHU State

Manages the list of Air Handling Units. Each AHU has identity fields (`id`, `name`, `tag`), a `type` (Recirculating, DOAS, MAU, FCU), capacity and airflow fields, and psychrometric override fields (`adp`, `adpMode`, `bypassFactor`).

4.2.1 AHU Types

`Recirculating` Standard recirculating AHU with partial outdoor air.

`DOAS` Dedicated Outdoor Air System—100% OA always.

`MAU` Make-up Air Unit—100% OA, typically for laboratories.

`FCU` Fan Coil Unit—recirculating, no OA path.

The logic layer distinguishes only DOAS vs non-DOAS (`isDOAS = ahuType === 'DOAS'`).

4.2.2 Reducers

`addAHU`, `updateAHU`, `deleteAHU`. **Warning:** `deleteAHU` must never be called directly from UI code—use `deleteAhuWithCleanup` thunk from `roomActions.js`.

4.3 `features/room/roomSlice.ts` — Room State

4.3.1 State Shape

```
state.room = {
  activeRoomId: string | null,
  list: Room[]  // see Room interface in types.ts
}
```

Listing 4.2: Room State Shape

Each `Room` contains: identity fields, geometry in SI (m^2, m^3), design targets (`designTemp` in °C, `designRH` in %, `pressure` in Pa), ISO classification, ventilation category, ACPH constraints, exhaust air breakdown (general, BIBO, machine in CFM), and AHU assignment.

4.3.2 Key Behaviours

- `designDB` (°F) is derived from `designTemp` (°C) and kept in sync by `updateRoom`.
- `classInOp` changes auto-update `minAcph` and `designAcph` from `ACPH_RANGES`.
- Geometry fields (`length`, `width`, `height`) auto-derive `floorArea` and `volume`.
- `designRH = 0` is valid (battery dry rooms)—the `!= null` guard preserves zero.
- `deleteRoom` never removes the last room.

4.4 `features/climate/climateSlice.ts` — Climate State

Stores outdoor design conditions for three seasons. All temperatures in °F.

```
state.climate.outside = {
  summer:  { db: 109.9, rh: 19, time, month, gr, dp, wb },
  monsoon: { db: 95,    rh: 70, ... },
  winter:  { db: 45,    rh: 60, ... }
}
```

Listing 4.3: Climate State Shape

Fields `gr`, `dp`, `wb` are display-only derived fields computed at sea level (elevation = 0). The calculation layer recomputes these with actual site elevation.

Summer default: 109.9°F (43.3°C)—ASHRAE HOF 2021 Ch.14 Table 1, 0.4% design dry-bulb for Delhi (28°N), correct tier for 24/7 critical facilities.

4.5 `features/envelope/envelopeSlice.ts` — Envelope State

Per-room envelope data stored in `state.envelope.byRoomId[roomId]`.

4.5.1 Structure

```
RoomEnvelope = {
  elements: {
    walls: [], roofs: [], glass: [],
    skylights: [], partitions: [], floors: []
  },
  internalLoads: {
    people:    { count, sensiblePerPerson, latentPerPerson },
    lights:    { wattsPerSqFt, useSchedule, ballastFactor },
    equipment: { kw, sensiblePct, latentPct, diversityFactor }
  },
  infiltration: {
    method: 'ach', achValue: 0, cfmValue: 0, doors: []
  }
}
```

Listing 4.4: Room Envelope Structure

`achValue` defaults to 0 (positively pressurised ISO/GMP rooms have zero infiltration by definition).

4.5.2 Reducers

`initializeRoom` (ISO-aware, guards against re-initialisation), `addEnvelopeElement`, `updateEnvelopeElement`, `removeEnvelopeElement`, `updateInternalLoad` (merge-update), `updateInfiltration`, `removeRoomEnvelope`.

Chapter 5

Cross-Slice Thunks

5.1 features/room/roomActions.js

Three thunks for atomic cross-slice operations:

addNewRoom() Creates a room in roomSlice and initialises its envelope in envelopeSlice atomically. Uses DEFAULT_NEW_ROOM_CLASS = 'ISO 8' as the single source of truth for both classification and ACPH defaults.

deleteRoomWithCleanup(roomId) Removes room from roomSlice and its envelope from envelopeSlice.

deleteAhuWithCleanup(ahuId) Clears all room assignments (setRoomAhu({ahuId: null})) *before* removing the AHU from ahuSlice. Order matters: clear first, then remove.

Chapter 6

Constants & Reference Data

6.1 constants/ashrae.ts — ASHRAE Constants

Central repository of ASHRAE Handbook constants. Exported as a frozen `as const` object. Key groups:

Table 6.1: Key ASHRAE Constants

Constant	Value	Source / Notes
SENSIBLE_FACTOR_SEA_LEVEL	1.08	$Q_s = 1.08 \times \text{CFM} \times \Delta T$
LATENT_FACTOR_SEA_LEVEL	0.68	$Q_l = 0.68 \times \text{CFM} \times \Delta\text{gr/lb}$
LATENT_FACTOR_LB	4775	$= 0.075 \times 60 \times 1061$
LATENT_HFG_BTU_LB	1061	h_{fg} at 32°F reference
BTU_PER_TON	12000	1 TR = 12,000 BTU/hr
M2_TO_FT2	10.7639	NIST exact
PROCESS_SAFETY_FACTOR	1.25	GMP Annex 1:2022 §4.23
PROCESS_DIVERSITY_FACTOR	0.75	Fallback only

Warning: Never use SENSIBLE_FACTOR_SEA_LEVEL or LATENT_FACTOR_SEA_LEVEL directly in calculations. Use sensibleFactor(elev) / latentFactor(elev) from psychro.ts.

6.2 constants/isoCleanroom.ts — ISO 14644 Classification

Contains:

- ISO_PARTICLE_LIMITS — ISO 14644-1:2015 particle concentration limits for ISO 1–9.
- ACPH_RANGES — IEST-RP-CC012.2 air change rates (min, design, max, OA fraction, flow type).
- ISO_CLASS_DATA — Full classification data including GMP grade mapping, typical use, pressure, temperature, and humidity ranges.

- `GMP_GRADE_MAPPING` — GMP Annex 1:2022 Grade A–D with ISO at-rest/in-operation requirements and mandated ACPH.

6.3 `constants/ventilation.ts` — ASHRAE 62.1-2022 VRP

Implements the Ventilation Rate Procedure with 12 industry-specific categories:

Table 6.2: Ventilation Categories (Selected)

Key	Label	R_p	R_a	Min ACH
`general`	General / Office	5	0.06	0
`pharma`	Pharma Cleanroom	5	0.18	20
`semicon`	Semiconductor Fab	5	0.18	6
`battery`	Battery — Li-ion	5	0.18	10
`battery-leadacid`	Battery — Lead-Acid	5	1.00	12
`gowning`	Gowning Room	5	0.10	10
`canteen`	Canteen / Cafeteria	7.5	0.18	0

Key exports: `calculateVbz()` (zone OA: $V_{bz} = R_p \times P_z + R_a \times A_z$) and `calculateMinAchCfm()` (regulatory ACH floor).

Chapter 7

Psychrometric Engine

7.1 `utils/psychro.ts` — Core Psychrometric Functions

The psychrometric engine uses **ASHRAE Hyland-Wexler** saturation pressure equations (HOF 2021 Ch.1, Eq. 3 for ice, Eq. 5 for liquid water), replacing the less accurate Magnus approximation used in earlier versions.

7.1.1 Saturation Pressure

$$\ln(P_{ws}) = \frac{C_1}{T} + C_2 + C_3 T + C_4 T^2 + C_5 T^3 + C_6 T^4 + C_7 \ln(T) \quad \text{(ice, } T < 273.15 \text{ K)} \tag{7.1}$$

$$\ln(P_{ws}) = \frac{C_8}{T} + C_9 + C_{10} T + C_{11} T^2 + C_{12} T^3 + C_{13} \ln(T) \quad \text{(liquid, } T \geq 273.15 \text{ K)} \tag{7.2}$$

7.1.2 Public API

Function	Description
`altitudeCorrectionFactor(elevFt)`	$C_f = (1 - 6.8754 \times 10^{-6} \times \text{elev})^{5.2559}$
`sitePressure(elevFt)`	$P_{\text{atm}} = 1013.25 \times C_f$ (hPa)
`sensibleFactor(elevFt)`	$C_s = 1.08 \times C_f$
`latentFactor(elevFt)`	$C_l = 0.68 \times C_f$
`calculateGrains(dbF, rh, elevFt)`	$W = 0.62198 \times E/(P_{\text{atm}} - E)$ (gr/lb)
`calculateRH(dbF, grains, elevFt)`	Reverse of `calculateGrains`
`calculateDewPoint(dbF, rh)`	Bisection on H-W curve; returns null for RH ≤ 0
`grainsFromDewPoint(dpF, elevFt)`	W from dew/frost point temperature

Function	Description
`calculateEnthalpy(dbF, grains)`	$h = 0.240t + W(h_{fg_0} + 0.444t)$
`calculateWetBulb(dbF, rh, elevFt)`	Bisection on ASHRAE Eq. 35 (± 0.01°C)
`calculateSpecificVolume(dbF, gr, elevFt)`	$v = 0.370486 \times T_R / P_{\text{psia}} \times (1 + 1.608W)$
`calculateAdpFromLoads(...)`	Back-calculates ADP from room sensible load
`calculateRequiredADP(...)`	ESHF line / saturation curve intersection

7.1.3 Required ADP — ESHF Analysis

Three possible outcomes:

`found` Bisection succeeded. `requiredADP` is the coil surface temperature needed. Compare with plant ADP.

`sensible_only` Required ADP < 32°F or ESHF $\geq 99.5\%$. Any standard CHW coil works.

`no_solution` ESHF line does not intersect saturation curve. Supplemental dehumidification required.

7.2 `utils/units.ts` — Unit Conversion Library

Pure, zero-dependency conversion library. Organised by family: area, volume, temperature, energy/power, humidity, airflow, pressure, mass flow, and length.

Key design decisions:

- Temperature conversions use `numOrNull` guard (returns `null` for invalid input) because 0°F is a valid temperature.
- Positive-value conversions (area, airflow) use `num()` guard (returns 0 for invalid input).
- Includes `PSYCHRO_SANITY_CHECKS` reference values for regression testing.

Chapter 8

Calculation Modules

8.1 `features/results/seasonalLoads.ts` — Seasonal Load Calculation

Computes per-room, per-season sensible and latent loads (BTU/hr).

8.1.1 Load Components

Sensible loads:

1. **Envelope** — CLTD/CLF method via `envelopeAggregator.ts`
2. **People** — sensible/person × count (HOF Ch.18 Table 1)
3. **Lighting** — W/ft^2 × area × 3.412 × schedule × ballast
4. **Equipment** — kW × 3412.14 × sensible% × diversity
5. **Infiltration** — $C_s \times \text{CFM} \times \Delta T$

Latent loads:

1. People latent heat
2. Equipment latent fraction
3. Infiltration — $C_l \times \text{CFM} \times \Delta\text{gr/lb}$

8.1.2 Safety Factor Policy

$$\text{ERSH} = \text{rawSensible} \times \underbrace{\left(1 + \frac{\text{safety} + \text{ductGain}}{100}\right)}_{\text{safetyMult}} \times \underbrace{1.25}_{\text{GMP rooms only}} \tag{8.1}$$

$$\text{ERLH} = \text{rawLatent} \quad \text{(no safety factor on latent)} \tag{8.2}$$

8.2 `features/results/airQuantities.ts` — Airflow Calculations

8.2.1 Supply Air Hierarchy

$$\text{supplyAir} = \max(\text{thermalCFM}, \text{designACPH\_CFM}, \text{regulatoryACPH\_CFM}, \text{minACPH\_CFM}) \tag{8.3}$$

Where:

$$\text{thermalCFM} = \frac{\text{ERSH}}{C_s \times (1 - BF) \times (T_{\text{room}} - T_{\text{ADP}})} \tag{8.4}$$

$$\text{designACPH\_CFM} = V_{\text{ft}^3} \times \text{designACPH}/60 \tag{8.5}$$

$$\text{regulatoryACPH\_CFM} = \texttt{calculateMinAchCfm}(\text{ventCategory}, V_{\text{ft}^3}) \tag{8.6}$$

8.2.2 Fresh Air (ASHRAE 62.1 VRP)

$$V_{bz} = R_p \times P_z + R_a \times A_z \tag{8.7}$$

Exhaust compensation: $\text{freshAir} = \max(V_{bz}, \text{totalExhaust})$.

For DOAS units: freshAir = supplyAir (100% OA).

8.2.3 Mass Balance

$$\text{returnAir} = \max(0, \text{supplyAir} - \text{freshAirCheck}) \tag{8.8}$$

8.3 `features/results/rdsSelector.ts` — RDS Orchestrator

The master memoised selector (`createSelector`) that assembles the complete RDS row for every room. Pure orchestrator—delegates all calculations:

1. **Step 1:** Seasonal loads (3 seasons × each room)
2. **Step 2:** Air quantities (supply, fresh, exhaust, return)
3. **Step 3:** Outdoor air coil loads (enthalpy method)
4. **Step 4:** Peak cooling season selection + grand total
5. **Step 4b:** Required ADP from ESHF line analysis
6. **Step 4c:** Reheater requirement (ASHRAE Ch.18 §17.3)
7. **Step 5:** Heating + humidification sizing
8. **Step 6:** CHW/HW pipe sizing
9. **Step 7:** Psychrometric state points
10. **Step 8:** Derived seasonal fields (pickup, terminal heating)

8.3.1 Reheater Logic

When roomESHF < minESHF, a reheater is required:

$$\text{minESHF} = \frac{C_s \times DR}{C_s \times DR + C_l \times \max(0, W_{\text{room}} - W_{\text{sat}}(T_{\text{ADP}}))} \tag{8.9}$$

$$\text{reheatBTU} = \frac{\text{minESHF} \times \text{ERTH}_{\text{safe}} - \text{ERSH}_{\text{safe}}}{1 - \text{minESHF}} \tag{8.10}$$

8.3.2 ADP Resolution Priority

1. AHU mode = `calculated` → `calculateAdpFromLoads()`
2. `ahu.adp > 0` → per-AHU manual override
3. `systemDesign.adp` → project default
4. 55°F hardcoded fallback

Chapter 9

Envelope Calculation Modules

9.1 utils/envelopeCalc.ts

CLTD/CLF-based heat gain calculations for opaque building elements:

- `calcWallGain(element, climate, tRoom, season)` — Wall conduction with CLTD, latitude/month correction, and daily range correction.
- `calcRoofGain(element, climate, tRoom, season)` — Roof conduction via flat-roof CLTD tables.
- `calcPartitionGain(element, tRoom, season)` — Steady-state conduction $Q = U \times A \times \Delta T$ with season-dependent adjacent temperatures.
- `calcInfiltrationGain(inf, room, volFt3, ...)` — Sensible and latent infiltration loads.

9.2 utils/glazingCalc.ts

Solar heat gain through glass and skylights:

- Conduction: $Q_c = U \times A \times \text{CLTD}_{\text{glass}}$
- Solar: $Q_s = A \times \text{SHGF} \times \text{SHGC} \times \text{CLF}$
- Total: $Q = Q_c + Q_s$

SHGC preferred over SC (SHGC $= 0.87 \times$ SC).

9.3 utils/envelopeAggregator.ts

Sums gains across all element categories for one room in one season. Calls individual `calcXxxGain()` functions from `envelopeCalc.ts` and `glazingCalc.ts`.

9.4 features/envelope/BuildingShell.tsx

Interactive tabbed UI for editing building envelope elements. Six categories (walls, roofs, glass, skylights, partitions, floors) with per-element ASHRAE preset selection, U-value input, area input, and per-season BTU/hr gain badges.

Chapter 10

Utility Modules

10.1 utils/types.ts — TypeScript Interfaces

Defines all shared interfaces:

Table 10.1: Key TypeScript Interfaces

Interface	Purpose
`Room`	Room identity, geometry, design targets, classification
`RoomState`	`{activeRoomId, list: Room[]}`
`AHU`	AHU equipment specification
`AhuState`	`{list: AHU[]}`
`ProjectInfo`	Project metadata
`AmbientParams`	Site reference conditions
`SystemDesign`	HVAC system sizing parameters
`ProjectState`	`{info, ambient, systemDesign}`
`EnvelopeElement`	Building element with U-value, area, orientation
`InternalPeople`	Occupant heat gain parameters
`InternalLights`	Lighting heat gain parameters
`InternalEquipment`	Equipment heat gain parameters
`RoomInfiltration`	Infiltration method and values
`RoomEnvelope`	Elements + internal loads + infiltration
`EnvelopeState`	`{byRoomId: Record<string, RoomEnvelope>}`
`SeasonCondition`	Outdoor condition for one season
`ClimateState`	Three-season outdoor conditions

`RootState` is re-exported from `store.ts` via `ReturnType<typeof store.getState>` for automatic type inference.

10.2 utils/isoValidation.ts

ISO 14644 and GMP compliance validation. Checks room classification consistency, ACPH sufficiency, pressure cascade compliance, and GMP grade mapping validity.

10.3 utils/psychroValidation.ts

Psychrometric consistency checks. Validates that room humidity targets are achievable given the selected ADP and bypass factor.

Chapter 11

UI Components

11.1 Layout Components

`components/Layout/Header.tsx` Application header bar with project branding.

`components/Layout/HeaderBadge.tsx` ASHRAE standard compliance badge.

`components/Layout/TabNav.tsx` Horizontal tab navigation (Project, RDS, AHU, Room, Climate, Envelope, Results).

`components/Layout/RoomSidebar.tsx` Vertical room list sidebar for multi-room navigation.

`components/Layout/RoomSidebarItem.tsx` Individual room entry in sidebar.

11.2 UI Primitives

`components/UI/InputField.tsx` Labelled text/number input with validation state.

`components/UI/InputGroup.tsx` Grouped inputs with shared label.

`components/UI/NumberControl.tsx` Numeric stepper with increment/decrement buttons.

`components/UI/StatCard.tsx` Display card for computed statistics.

11.3 `features/room/AhuAssignment.jsx`

Radio-button single-select AHU assignment component. Replaces the previous multi-select checkbox UI that was architecturally inconsistent—all downstream consumers read only `assignedAhuIds[0]`.

Chapter 12

Custom Hooks

`hooks/useActiveRoom.js` Returns the currently selected room object and dispatch helpers for room field updates.

`hooks/useNumberControl.ts` Manages numeric input state with increment/decrement, min/max bounds, and step size.

`hooks/useProjectTotals.js` Aggregates RDS data across all rooms into project-level totals (total TR, total CFM, per-AHU summaries).

`hooks/useRdsRow.js` Returns the full computed RDS row object for a single room, including dispatch helpers for inline editing.

`hooks/useRoomSidebar.ts` Manages room sidebar state (active room, add/delete room, room ordering).

Chapter 13

Pages

13.1 `pages/Home.jsx` — Landing Page

Public-facing landing page without the application header/nav. Entry point for new projects.

13.2 `pages/ProjectDetails.jsx` — Project Configuration

Three sections: project metadata (name, location, customer), site ambient conditions (elevation, latitude, daily range), and system design parameters (safety factor, BF, ADP, fan heat).

13.3 `pages/AHUConfig.jsx` — AHU Equipment

Add, edit, and delete AHUs. Fields: type (Recirculating/DOAS/MAU/FCU), capacity, airflow, psychrometric overrides, filter class, location, notes.

13.4 `pages/RoomConfig.jsx` — Room Setup

Left column: room identity, geometry, design targets, ISO classification, ventilation category, ACPH settings, exhaust air breakdown. Right column: AHU assignment.

13.5 `pages/ClimateConfig.jsx` — Outdoor Conditions

Three-season (summer, monsoon, winter) outdoor design conditions. Each season: DB, RH, time of peak, month. Derived fields (gr, dp, wb) shown as read-only.

13.6 `pages/EnvelopeConfig.jsx` — Building Envelope

Hosts the `BuildingShell` component. Six categories of envelope elements with ASHRAE construction presets and per-season BTU/hr gain display.

13.7 `pages/RDSPage.jsx` — Room Data Sheet

Spreadsheet-style view of all computed RDS data. Groups rooms by AHU. Sections: setup, loads, psychrometric state points, results.

13.8 `pages/ResultsPage.jsx` — Project Results

Project-level aggregation: total cooling capacity (TR), total heating capacity (kW), per-AHU summaries, check figure (ft^2/TR), and Excel export.

Chapter 14

Deployment

14.1 Build & Deploy

```
cd client
npm install
npm run build     # Production bundle
npm run dev       # Development server with HMR
npm run test      # Vitest test suite
```

Listing 14.1: Build Commands

14.2 Vercel Configuration

SPA rewrite rule in `vercel.json`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Listing 14.2: Vercel Rewrite Rule

All client-side routes are handled by React Router; the server always serves `index.html`.

Chapter 15

Engineering Standards Reference

Standard	Usage in Application
ASHRAE HOF 2021, Ch.1	Psychrometric equations (Hyland-Wexler)
ASHRAE HOF 2021, Ch.18	CLTD/CLF load calculation methodology
ASHRAE 62.1-2022	Ventilation Rate Procedure (VRP)
ASHRAE 90.1-2022	Energy standard (lighting power density)
ISO 14644-1:2015	Cleanroom classification (particle limits)
ISO 14644-4:2022	Cleanroom design & construction
GMP Annex 1:2022	Pharmaceutical HVAC requirements
IEST-RP-CC012.2	Cleanroom air change rates
NFPA 855	Battery energy storage systems
IFC §1206	Fire code — battery rooms
OSHA 29 CFR 1926.403(i)	Lead-acid battery charging ventilation
IEEE 1184-2006	Battery installation sizing
SEMI S2-0200	Semiconductor facility safety
ISPE Baseline Guide Vol.5	Pharmaceutical cleanroom HVAC

Chapter 16

Appendix: Key Equations

16.1 Sensible Load

$$Q_s = C_s \times \text{CFM} \times \Delta T \quad \text{[BTU/hr]} \tag{16.1}$$

where $C_s = 1.08 \times C_f$ and $C_f = (1 - 6.8754 \times 10^{-6} \times \text{elev}_{\text{ft}})^{5.2559}$.

16.2 Latent Load

$$Q_l = C_l \times \text{CFM} \times \Delta W \quad \text{[BTU/hr]} \tag{16.2}$$

where $C_l = 0.68 \times C_f$ and ΔW is in gr/lb.

16.3 Humidity Ratio

$$W = \frac{0.62198 \times E}{P_{\text{atm}} - E} \quad \text{[kg/kg]} \tag{16.3}$$

where $E = (\text{RH}/100) \times P_{ws}(T)$ and P_{ws} is from Hyland-Wexler.

16.4 Enthalpy

$$h = 0.240 \cdot t + W \cdot (1061 + 0.444 \cdot t) \quad \text{[BTU/lb dry air]} \tag{16.4}$$

16.5 Ventilation Zone Breathing Rate

$$V_{bz} = R_p \times P_z + R_a \times A_z \quad \text{[CFM]} \tag{16.5}$$

16.6 Thermal Supply Air

$$\text{CFM}_{\text{thermal}} = \frac{\text{ERSH}}{C_s \times (1 - BF) \times (T_{\text{room}} - T_{\text{ADP}})} \tag{16.6}$$

16.7 Effective Room Sensible Heat Factor

$$\text{ESHF} = \frac{Q_s^{\text{total}}}{Q_s^{\text{total}} + Q_l^{\text{total}}} \tag{16.7}$$

16.8 Reheater Capacity

$$Q_{\text{reheat}} = \frac{\text{minESHF} \times \text{ERTH} - \text{ERSH}}{1 - \text{minESHF}} \quad [\text{BTU/hr}] \tag{16.8}$$

16.9 Cooling Capacity

$$\text{TR} = \frac{\text{ERSH} + \text{ERLH} + Q_{\text{OA}} + Q_{\text{fan}} + Q_{\text{reheat}}}{12000} \tag{16.9}$$

End of Documentation — HeatLoad HVAC Calculation Tool v2.8